PROPOSAL ARENA
Simulasi Sidang Proposal Skripsi Berbasis AI

Dokumentasi Teknis untuk Presentasi Midtest
Oleh: Mahasiswa Semester 5 - Ilmu Komputer
Tanggal: 29 September 2026

1. PENDAHULUAN

1.1 Latar Belakang

Proposal Arena adalah aplikasi simulasi sidang proposal skripsi berbasis web yang menggunakan Artificial Intelligence (AI) sebagai pemeriksa virtual. Aplikasi ini dirancang untuk membantu mahasiswa dalam mempersiapkan sidang proposal aktual dengan menyediakan lingkungan simulasi yang realistis dan interaktif.

Aplikasi ini dibangun dengan pendekatan client-side yang sepenuhnya berjalan di browser, tanpa memerlukan server backend, database server, atau infrastruktur server lainnya. Pendekatan ini membuat aplikasi dapat di-deploy secara statis dan mudah diakses dari berbagai perangkat.

1.2 Tujuan Pengembangan

- Menyediakan platform simulasi sidang proposal yang dapat diakses secara lokal
- Memanfaatkan AI sebagai examiner virtual yang adaptif dan responsif
- Menjaga data pengguna tetap lokal di browser (privacy-first design)
- Men-deploy aplikasi secara statis tanpa memerlukan backend server
- Memberikan pengalaman simulasi yang mendekati sidang proposal sungguhan

2. ARSITEKTUR TEKNIS

2.1 Stack Teknologi

Aplikasi ini dibangun menggunakan kombinasi teknologi modern berikut:

Kategori	Teknologi	Versi	Kegunaan
Framework	Astro + TypeScript	Latest	Static site generation + type safety
UI Components	Svelte	Latest	Komponen interaktif defense simulation
Styling	Tailwind CSS	3.4.0	Responsive design system
State Management	nanostores	1.5.3	Reactive state untuk Svelte
Persistence	@nanostores/persistent	1.3.5	State persistence di localStorage
Local Storage	IndexedDB (idb)	8.0.3	Penyimpanan data proposal & session
PDF Processing	pdfjs-dist	6.3.289	Ekstraksi teks dari file PDF
Search Engine	MiniSearch	7.2.0	Full-text search untuk grounding
Internationalization	svelte-i18n	4.0.1	UI localization (EN/ID)
AI Provider	OpenRouter/OpenAI/Gemini	-	LLM integration abstraction
Voice	Web Speech API	Browser	Text-to-speech untuk pertanyaan

2.2 Struktur Direktori

```
ppl-semprosimulator/
|-- src/
|   |-- App.svelte              # Root component
|   |-- components/
|   |   |-- eval/               # Halaman evaluasi manusia
|   |   |-- interview/          # Komponen defense simulation
|   |   |-- layout/             # Komponen layout (Header, Settings)
|   |   `-- upload/             # File upload component
|   |-- lib/
|   |   |-- ai/                 # AI pipeline & provider abstraction
|   |   |-- db/                 # IndexedDB wrapper
|   |   |-- file/               # File extractor & language detection
|   |   |-- interview/          # Controller & state machine
|   |   |-- search/             # MiniSearch indexer & chunker
|   |   |-- state/              # Nanostores state
|   |   |-- storage/            # Settings & localStorage
|   |   `-- voice/              # Web Speech API wrapper
|   |-- pages/                  # Astro pages
|   |-- prompts/                # AI prompt templates
|   `-- types/                  # TypeScript type definitions
|-- dist/                       # Production build output
`-- prompts/                    # Examiner persona prompt
```

2.3 Desain Terkunci (Design Decisions)

- Client-side only: Tidak ada backend server, VPS, PostgreSQL, Redis, atau S3
- User-provided API keys: Aplikasi tidak menyimpan rahasia project, pengguna yang menyediakan kuncinya
- Static deployable: Dapat di-host di GitHub Pages, Netlify, Vercel, atau platform statis lainnya
- Local-first data: Semua data tesis, analisis, transkrip, dan laporan tersimpan di browser pengguna
- AI provider abstraction: Logika fitur tidak bergantung pada provider tertentu, mudah ditukar
- Immutable proposal: Dokumen asli tidak dimodifikasi, data turunan disimpan terpisah

3. ALUR PEMROSESAN LLM

3.1 Gambaran Umum

Aplikasi ini menggunakan Large Language Model (LLM) melalui 4 fungsi utama yang masing-masing memiliki tujuan spesifik dalam simulasi defense. Semua komunikasi dengan LLM dilakukan melalui abstraction layer yang seragam.

3.2 Tahapan Pemrosesan

Tahap 1: Upload dan Ekstraksi File

```
FileUploader.svelte
   |
   v
extractTextFromFile() (src/lib/file/extractor.ts)
   |-- PDF: Gunakan pdfjs-dist untuk extract teks per halaman
   |-- TXT: Langsung baca sebagai teks
   |
   v
detectLanguage() (heuristic stopword-based)
   |-- Hitung kata kunci Bahasa Indonesia vs Inggris
   |-- Tentukan bahasa dominan (id/en)
   |
   v
Simpan ke IndexedDB sebagai Proposal record
```

Tahap 2: Analisis Proposal (analyzeProposal)

```
Pipeline: analyzeProposal()
Input:
  - provider: AIProvider instance
  - text: Full proposal text (truncated to 15,000 chars)
  - language: "en" atau "id"

Process:
  1. Construct prompt dengan schema JSON yang spesifik
  2. Set temperature = 0.1 (low = consistent output)
  3. Requested response_format = "json_object"

Output Schema (ProposalAnalysis):
  {
    "title": "String",
    "field": "Computer Science sub-field",
    "researchProblem": "String",
    "researchQuestions": ["String"],
    "objectives": ["String"],
    "methodology": "String",
    "limitations": "String",
    "weaknesses": "String",
    "likelyTopics": ["String"]
  }
Simpan ke IndexedDB (proposalAnalyses store)
```

Tahap 3: Generasi Pertanyaan (generateQuestion)

```
Pipeline: generateQuestion()
Input:
  - analysis: ProposalAnalysis (dari tahap 2)
  - transcript: Array InterviewMessage (6 pesan terakhir)
  - language: "en" atau "id"
  - difficulty: 1-5 (dari session state)

Process:
  1. Ambil 6 pesan terakhir dari transcript
  2. Search MiniSearch index untuk relevant context chunks
     - Query: transkrip teks terakhir
     - Filter: proposalId, fuzzy search, prefix matching
  3. Load examiner.md prompt template
  4. Replace placeholders:
     {{LANGUAGE}}      -> "Bahasa Indonesia" atau "English"
     {{DIFFICULTY}}    -> "1" sampai "5"
     {{FIELD}}         -> misal "Machine Learning"
```

```
    {{ANALYSIS}}      -> JSON analysis (indented)
    {{RELEVANT_CONTEXT}} -> chunks teks relevan
    {{TRANSCRIPT}}    -> pesan examiner & mahasiswa
  5. Set temperature = 0.7 (lebih kreatif untuk variasi)
  6. Send ke LLM

Output: String (satu pertanyaan examiner)
```

Tahap 4: Evaluasi Jawaban (evaluateAnswer)

```
Pipeline: evaluateAnswer()
Input:
  - question: String (pertanyaan examiner)
  - answer: String (jawaban mahasiswa)
  - language: "en" atau "id"

Output Schema (AnswerEvaluation):
  {
    "correctness": 1-5,      // Seberapa benar jawaban
    "relevance": 1-5,        // Relevansi dengan pertanyaan
    "depth": 1-5,            // Kedalaman analisis
    "justification": 1-5,    // Kualitas justifikasi
    "clarity": 1-5,          // Kejelasan ekspresi
    "feedback": "String"     // Saran perbaikan konstruktif
  }

Catatan: Saat ini hasil eval dicetak ke console.log
tetapi BELUM disimpan ke IndexedDB (perlu pengembangan)
```

Tahap 5: Laporan Akhir (generateReport)

```
Pipeline: generateReport()
Input:
  - transcript: Array InterviewMessage (seluruh percakapan)
  - language: "en" atau "id"

Output Schema (DefenseReport):
  {
    "strengths": ["String"],           // Kelebihan mahasiswa
    "weaknesses": ["String"],          // Kekurangan yang perlu diperbaiki
    "recommendations": ["String"],     // Saran untuk perbaikan
    "practiceQuestions": ["String"],   // Pertanyaan latihan tambahan
    "notes": ["String"],               // Feedback gaya defense
    "summary": "String"                // Ringkasan keseluruhan
  }
```

3.3 Diagram Alur Lengkap

```
ALGORITMA UTAMA (controller.ts):

initSession(proposalId)
    |-- Load proposal dari IndexedDB
    |-- Buat session baru (random UUID)
    |-- Set state = "READY"
    |-- Index proposal ke MiniSearch (globalIndexer.indexProposal())

LOOP DEFENSE:
  askNextQuestion()
    |-- getProvider()          -> Buat AIProvider dari settings
    |-- dbStore.getMessages()  -> Load transkrip
    |-- generateQuestion()     -> LLM generate pertanyaan
    |-- addMessage()           -> Save ke IndexedDB
    |-- speak()                -> Text-to-speech (jika diaktifkan)
    |-- Set state = "WAITING"
    |-- Update session di IndexedDB

  submitAnswer(answer)
    |-- addMessage()           -> Save jawaban user
    |-- Set state = "EVALUATING"
    |-- generateQuestion()     -> Temukan last question
    |-- evaluateAnswer()       -> LLM evaluate jawaban
    |     (hasil ke console.log, BELUM ke IndexedDB)
    |-- askNextQuestion()      -> Auto-advance ke pertanyaan berikutnya
    |-- Set state = "WAITING"

    [LOOP back ke askNextQuestion()]

endInterview()
    |-- Set state = "REPORT_GENERATION"
    |-- generateReport()       -> LLM generate laporan
    |-- dbStore.saveReport()   -> Simpan ke IndexedDB
    |-- Set state = "REPORT_READY"
    |-- Redirect ke /report?id=<sessionId>
```

4. PROMPT ENGINEERING & PERSONA AI

4.1 Persona Examiner

AI examiner bertindak sebagai profesor bidang Computer Science yang berpengalaman, ketat namun adil. Berikut adalah rincian persona yang diimplementasikan dalam prompt template (src/prompts/examiner.md):

- Identitas: Profesor CS dengan keahlian di software engineering, algorithms, system architecture, AI
- Tingkat kesulitan adaptif (1-5): Dari pertanyaan definisi surface-level hingga probing edge cases
- Aturan interaksi: Hanya satu pertanyaan utama per respons, nada conversational & mentoring
- Progresi topik: Problem & motivation -> Approach/architecture -> Implementation -> Evaluation -> Limitations
- Proposal anchoring: Setiap pertanyaan harus langsung berkaitan dengan proposal yang diupload
- Konsistensi bahasa: Defense dilakukan sepenuhnya dalam bahasa yang terdeteksi dari proposal
- Guardrails: Menolak permintaan mahasiswa untuk menulis kode, menjaga fokus pada defense

4.2 Tingkat Kesulitan (Difficulty Levels)

Level	Fokus Pertanyaan	Ekspektasi	Contoh
1	Definisi, scope, core architecture	Terima jawaban impresis/incomplete	Apa masalah utama yang akan diselesaikan?
2	Implementasi details, tools, datasets	Harapkan beberapa spesifisitas	Bagaimana pipeline dasar sistem kamu?
3	Design choices dan trade-offs	Coba defense pilihan method A vs B	Kenapa pilih pendekatan ini vs pendekatan lain?
4	Evaluation rigor, metrics, validity	Tekan spesifisitas testing	Metrics apa untuk mengukur keberhasilan?
5	Edge cases, limitations, scalability	No hand-waving, tekan batasan	Bagaimana sistem menangani beban besar/edge cases?

4.3 Strategi Prompt

- JSON response format: Untuk operasi internal, gunakan response_format: "json_object" dengan temperature rendah (0.1-0.2)
- Template substitution: examiner.md menggunakan template literal dengan placeholder untuk parameterisasi
- Grounding dengan MiniSearch: generateQuestion mencari cuplikan relevan dari proposal sebelum membentuk pertanyaan
- Temperature control: 0.1 untuk analisis & evaluasi (deterministic), 0.7 untuk pertanyaan (variatif), 0.2 untuk laporan
- System prompt differentiation: Setiap fungsi menggunakan system prompt yang berbeda sesuai peran AI

5. DATABASE & PERSISTENCE

5.1 IndexedDB Schema

Aplikasi menggunakan IndexedDB melalui library `idb` untuk penyimpanan data:

Object Store	Key Path	Indexes	Keterangan
proposals	id	-	Proposal teks lengkap, judul, bahasa, waktu upload
proposalAnalyses	id	-	Analisis LLM: field, masalah, metodologi, kelemahan
interviewSessions	id	-	State machine, questionIndex, difficulty, timestamps
interviewMessages	id	sessionId	Transkrip: role, content, timestamp per pesan
answerEvaluations	id	-	Skor LLM: correctness, relevance, depth, feedback
reports	id	-	DefenseReport lengkap setelah session selesai
judgments	id	sessionId	HumanJudgment: relevance/difficulty score dari manusia

5.2 localStorage Penggunaan

- API key: Disimpan sementara di memory, tidak persist di localStorage (security)
- UI language preference: Disimpan di localStorage melalui @nanostores/persistent
- Settings konfigurasi: Provider ID, model pilihan, preferensi voice
- Catatan: localStorage digunakan ONLY untuk preferensi UI kecil, bukan data sensitif

5.3 MiniSearch Indexing

```
ProposalIndexer (src/lib/search/indexer.ts):

1. indexProposal(proposalId, text)
   |-- chunkText(text) -> potong teks menjadi chunks kecil
   |-- Buat ChunkRecord untuk setiap chunk:
   |     { id: "proposalId_chunk_0", proposalId, text }
   |-- this.minisearch.addAll(records)

2. searchContext(proposalId, query, maxResults=3)
   |-- Bersihkan query (remove special chars)
   |-- this.minisearch.search(query, {
   |     filter: (result) => result.proposalId === proposalId,
   |     combineWith: "OR",
   |     prefix: true,
   |     fuzzy: 0.1
   |   })
   |-- Return top 3 chunks sebagai array string
```

6. COMPONENTS & INTERFACE

6.1 Komponen Svelte Utama

Komponen	Fungsi	Fitur Kunci
App.svelte	Root component, routing tampilan	Mengatur tampilan utama berdasarkan state
SplitLayout.svelte	Layout dua panel	Kiri: dokumen, Kanan: chat defense
FileUploader.svelte	Drag-and-drop upload	Support PDF/TXT, extract text, detect language
SettingsModal.svelte	Konfigurasi API key & provider	Pilih provider, set model, toggle voice
ChatWindow.svelte	Menampilkan transkrip defense	Scrollable chat, loading state, voice feedback
Controls.svelte	Input teks & voice dictation	Text input, send button, voice toggle, dictation
IntegrityMonitor.svelte	Deteksi tab switch/window blur	Peringatan jika meninggalkan tab selama defense
ReportViewer.svelte	Tampilkan DefenseReport	Strengths, weaknesses, recommendations, practice Qs
Dashboard.svelte	Evaluasi manusia dashboard	Tabel session, skor pertanyaan, input judgment

6.2 Halaman Astro

- index.astro: Halaman utama, menampilkan App.svelte dengan Form Upload + Settings Modal
- eval.astro: Halaman dashboard evaluasi manusia (evaluator/instructor)
- report.astro: Halaman laporan defense (diakses setelah endInterview)

6.3 State Machine (InterviewState)

```
InterviewState (src/lib/interview/engine.ts):

CREATED --> READY --> QUESTIONING --> WAITING_FOR_ANSWER
                                          |
                                          v
                                     EVALUATING <---+
                                          |     |
                                     +----------+     |
                                     v                |
                                FOLLOW_UP ---------------+
                                     |
                                     v
                                WAITING_FOR_ANSWER (loop)

EVALUATING --> COMPLETED --> REPORT_GENERATION --> REPORT_READY
```

7. KEUNIKAN & FITUR KHUSUS

7.1 Fitur Voice (Web Speech API)

- Text-to-speech: Pertanyaan examiner diucapkan secara otomatis jika voice toggle diaktifkan
- Speech recognition: Mahasiswa dapat menjawab dengan voice dictation (microphone)
- Implementasi: Menggunakan Web Speech API bawaan browser (SpeechSynthesis & SpeechRecognition)
- Fallback: Jika browser tidak mendukung, fitur voice otomatis non-aktif dengan notifikasi

7.2 Deteksi Bahasa Otomatis

Setelah file berhasil diupload dan teks diekstrak, aplikasi secara otomatis mendeteksi bahasa proposal menggunakan heuristic sederhana berbasis stopwords:

```
function detectLanguage(text: string): "en" | "id" {
  const enWords = ["the", "is", "in", "and", "to", "of", "a", "for", "this", "that", "with"];
  const idWords = ["yang", "di", "dan", "ini", "untuk", "dari", "dalam", "dengan", "pada", "adalah"];

  // Hitung kemunculan kata kunci masing-masing bahasa
  // Return bahasa dengan jumlah kata kunci lebih banyak
}
```

Catatan: Deteksi ini bersifat heuristik dan cukup untuk proposal yang dominan dalam satu bahasa. Untuk proposal mixed-language, aplikasi menyediakan konfirmasi kepada pengguna sebelum defense dimulai.

7.3 Integritas Session (Anti-Cheating)

- IntegrityMonitor.svelte: Mendeteksi jika pengguna beralih tab atau windowBlur selama defense
- Peringatan tampil jika session integritas terganggu
- Data session tetap tersimpan, tetapi peringatan mencatat bahwa defense mungkin tidak valid

7.4 Fallback Model Chain

OpenAICompatibleProvider mendukung comma-separated model list untuk fallback chain. Jika model pertama gagal, provider akan mencoba model berikutnya.

```
Contoh konfigurasi model:
  "meta-llama/llama-3.1-8b-instruct:free,openrouter/nous-ai/nous-hermes-2-mixture-of-experts"

Flow:
  1. Coba model pertama -> Jika gagal, log warning
  2. Coba model kedua -> Jika berhasil, return response
  3. Jika semua gagal -> Throw error dengan pesan terakhir
```

8. KETERBATASAN & PENGEMBANGAN MASA DEPAN

8.1 Keterbatasan Saat Ini (Known Issues)

Berikut adalah keterbatasan yang diakui dalam versi saat ini:

- Evaluasi jawaban tidak tersimpan: submitAnswer() mencetak evalResult ke console.log tetapi tidak menyimpan ke IndexedDB - data evaluasi hilang setelah refresh
- Deteksi bahasa sederhana: Hanya menggunakan 10 kata kunci per bahasa, bisa salah untuk proposal mixed-language
- Tidak ada tombol restart: endInterview() langsung membuat laporan, tidak ada opsi restart session di tengah jalan
- evaluateAnswer() tanpa konteks proposal: Fungsi evaluasi hanya menerima question + answer, tidak ada ringkasan proposal
- Prompt tidak terpusat: analyzeProposal dan generateReport menggunakan template literal di pipeline.ts, bukan di file /prompts/ seperti examiner.md (melanggar pedoman AGENTS.md)
- Hardcoded UI strings: Banyak komponen masih menulis teks langsung dalam kode, melanggar aturan i18n
- No UX feedback saat loading: Proses analysis dan question generation bisa memakan waktu, loading spinner bisa lebih jelas

8.2 Pengembangan Masa Depan (Roadmap)

Prioritas	Fitur	Keterangan
High	Simpan evaluasi ke DB	Implement dbStore.saveEvaluation() dan tampilkan di report/dashboard
High	Centralize semua prompt	Pindahkan prompt-analyze dan prompt-report ke /prompts/ folder
High	Implementasi i18n lengkap	Gunakan svelte-i18n untuk semua UI strings, buat en.json & id.json
Medium	Deteksi bahasa lebih baik	Gunakan list stopwords lebih besar atau library language detection
Medium	Mobile responsive	Rearrang SplitLayout menjadi single-panel dengan tab/accordion
Medium	Tombol restart session	Ekspos ulang function initSession() di UI untuk restart
Low	Export/import data	Allow users to export session data sebagai JSON/PDF
Low	Multi-provider support	Tambahkan adapter untuk provider lain (Anthropic, Cohere, dll.)

9. KEAMANAN & PRIVASI

9.1 Model Keamanan

- User-provided API keys: Kunci API disediakan oleh pengguna, tidak disimpan dalam project atau source control
- No server-side proxy: Permintaan API dilakukan secara langsung dari browser ke provider, tanpa proxy server
- No logging: API key tidak dicetak ke console atau log, tidak dikirim ke layanan lain
- Local data storage: Semua data tesis, analisis, dan transkrip tersimpan di browser pengguna (IndexedDB)
- Clear data: Pengguna dapat menghapus data lokal kapan saja melalui browser dev tools atau reset aplikasi

9.2 Peringatan Pengguna

Sebelum mengirim proposal ke LLM, aplikasi menampilkan pesan peringatan:

```
"Catatan privasi: Teks proposal Anda akan dikirim ke AI provider
(OpenRouter/OpenAI/Gemini) untuk dianalisis. Pastikan Anda tidak
memasukkan informasi sensitif atau pribadi dalam proposal yang
anda upload."
```

Peringatan ini penting karena meskipun aplikasi tidak menyimpan data di server, teks proposal tetap dikirim ke API provider untuk diproses.

10. PENGEMBANGAN & DEPLOYMENT

10.1 Perintah Development

```
# Install dependencies
bun install

# Jalankan development server (hot reload)
bun run dev
# Aplikasi berjalan di http://localhost:4321

# Type checking
bun run check

# Production build
bun run build
# Output statis di folder dist/

# Preview build lokal
bun run preview
```

10.2 Konfigurasi Environment

```
# .env.example (jangan commit ke git!)
OPENROUTER_API_KEY=your_key_here
OPENAI_API_KEY=your_key_here
GEMINI_API_KEY=your_key_here

# Atau set di SettingsModal UI saat runtime
```

10.3 Deployment Option

- GitHub Pages: Push ke branch gh-pages atau gunakan GitHub Actions untuk build & deploy
- Netlify: Connect repository, build command: bun run build, publish directory: dist
- Vercel: Import project, build command: bun run build, output: dist
- Static hosting lain: Apache, Nginx, Cloudflare Pages, dll.
- Catatan: Tidak perlu backend server, semua process terjadi di client

10.4 Alur Kerja Git (Branch & PR)

Proyek ini menggunakan alur kerja Git yang baik untuk pengembangan tim:

- JANGAN push langsung ke branch main
- Buat branch baru untuk setiap fitur: git checkout -b fitur/nama-fitur
- Commit dengan pesan yang jelas: feat:, fix:, docs:, refactor:, test:
- Jalankan bun run check dan bun run build sebelum push
- Buat Pull Request untuk review sebelum merge ke main

PENUTUP

Proposal Arena adalah proyek pembelajaran yang menggabungkan pengembangan web modern dengan penggunaan AI sebagai alat bantu. Aplikasi ini mendemonstrasikan kemampuan dalam:

- Membangun aplikasi web full-stack client-side dengan Astro + Svelte + TypeScript
- Mengintegrasikan AI/LLM API dengan abstraction layer yang rapi
- Mengelola state aplikasi yang kompleks dengan nanostores dan IndexedDB
- Merancang prompt engineering untuk persona AI yang konsisten dan adaptif
- Mengimplementasikan fitur voice, file processing, dan full-text search
- Mempertimbangkan keamanan, privasi, dan lokal-first design

Aplikasi ini dirancang untuk membantu mahasiswa dalam mempersiapkan sidang proposal skripsi dengan lingkungan simulasi yang realistis, sekaligus sebagai proyek pengembangan yang mengajarkan best practice dalam software engineering modern.

Semoga simulasi sidang proposal yang dibuat membantu banyak mahasiswa dalam mempersiapkan sidang nyata.